

DDL-Center 作业报告

一、程序功能介绍

DDL 指挥中心是一款专为北大学生开发的本地桌面软件。它的核心目标是把散落在教学网、信息门户以及各种备忘录里的 DDL 整合并进行集中管理。借助这个软件，同学们可以一站式搞定作业、上课日程、考前复习安排和个人待办事项。

1. DDL 管理

(1) 基本操作：用户可以手动加任务、删任务、改状态。创建任务的时候可以自定义标题，将任务关联到相对应的课上，并设定完成时间、预估时长，以及高/中/低优先级。任务状态分为待办、在做、搞定和受阻这几种。

(2) 多视角浏览：支持按“时间轴”或者“课程分类”看任务；也支持按状态（全部/未完成/已完成/逾期）、课程、截止时间筛选和排列任务。

(3) 支持一键标记完成，系统自动记录完成时间。

2. 教学网自动同步

北大学生可通过北大 IAAA 统一身份认证登录教学网，程序会自动抓取全部待办作业入库。之后，已逾期的作业会被自动丢弃，有明确截止时间的将被直接导入，无明确截止时间的则交给 AI 助手解析。值得一提的是，同步已存在的任务时，用户手动更改过的部分将会自动保存，不会被同步命令覆盖。

3. 课程表与考试安排

北大学生可从门户导出 HTML 格式的课表，并使用一键导入功能将课表导入 DDL-Center。课程时间、地点、单双周、周次范围和期末考试信息都将被自动识别。应用支持单双周切换和 DDL 与课表同时显现。

4. 智能预警中心

(1) 分设 3 天/ 1 天/逾期三档截止提醒。

(2) 超载预警：当同一天 DDL 数量超过阈值，程序就会触发警报。

(3) 拥堵预警：当相邻两个 DDL 间隔不足 24 小时，程序也会触发警报。

5. 空闲时间计算与智能推荐

AI 根据课表占用时段反推每日空闲时段，结合任务预计耗时、优先级、截止时间等信息，依据整块时间优于碎片时间的原则推荐最合适的执行时段。

6. 进度可视化

将 matplotlib 绘制的图表嵌入 Qt 界面，直观展示完成率、工作量分布与负载状

态。

7. 内置 AI 助手

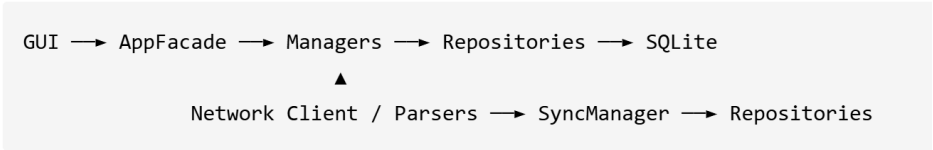
- (1) 任务智能拆解：将一个大任务拆解成带预计耗时的子任务。
- (2) 学习辅助对话：内嵌聊天助手，能自动获取任务信息并给出学习建议。
- (3) 每日简报：基于当天的任务和课表生成一句话简报。
- (4) 同步内容摘要：将教学网上原文的核心要素提炼出来。
- (5) 自然语言录入：输入一段自然语言，AI 助手能自动判断这是任务、课程还是考试，并自动创建为新的任务。

8. 个性化外观

4 套主题（PKU 红 / THU 紫 / 海蓝 / 深色）、4 档字号、自定义头像与显示名，均可依据用户喜好进行修改。

二、项目各模块与类设计细节

1. 总体架构



2. 模型层（app/models/）—— 纯数据类

这一部分承载业务实体，不涉及 IO 功能，只做纯计算判断。

类	关键属性	关键方法
Task	title / course_id / due_time / estimated_hours / status / priority / source / user_modified / external_id	mark_done() / is_overdue() / is_due_within() / days_left()
Course	name / teacher / semester / color / source	display_name() / has_external_id()
ScheduleSlot	weekday / start_time / end_time / location / slot_type / start_week / end_week / week_type	duration_hours() / occurs_in_week() / overlaps_with() / can_hold_task()
Exam	course_id / name / start_time / end_time / location / seat / exam_type	duration_hours() / is_upcoming() / days_left()
Alert	task_id / level / kind / message / is_read	is_urgent() / display_text()
SyncRecord	source_type / external_id / local_type / local_id / raw_hash / status	记录外部对象 ↔ 本地对象映射，支撑去重

3. Manager 层（app/managers/）—— 业务逻辑

(1) **AppFacade**: 这是整个后端的“总台”，聚合了全部业务方法（create_task / list_tasks / generate_alerts / list_schedule / recommend_for_task / sync_from_teaching_site / get_statistics 及 5 个 AI 方法 + 设置方法）。

(2) **TaskManager**: 支持任务的增加、读取、修改、删除和按课程、状态查询；负责在用户手动编辑时置 user_modified 为 1，若是同步教学网则保持其默认值 0，进而实现“同步不覆盖用户编辑”这一功能，并统一填充 created_at / updated_at / source 等字段。

(3) **AlertManager**: 用于预警；generate_deadline_alerts 实现 3 天/1 天/逾期提醒，detect_same_day_overload 实现超载预警，detect_close_deadlines 实现间隔 <24h 预警。

(4) **ScheduleManager**: 用于排日程；list_slots 实现完成任务时段安排，has_conflict 实现冲突检测，get_free_slots 实现空闲段计算。

(5) **ExamManager**: 用于管理考试进程；支持考试进程的增加、读取、修改、删除和列出未完成考试 list。

(6) **RecommendationManager**: 按“DDL 越靠前，高优先级优先，空闲段足够长，整块时段优先”的排序推荐任务执行时段。

(7) **StatisticsManager**: 展现完成率、进度曲线、负载状态和图表化数据。

(8) **SyncManager**: 实现教学网同步合并的关键。

- external_id 不存在，则为新增
- 存在且 raw_hash 不变，则跳过
- 存在且 raw_hash 变化且 user_modified=0，则更新
- 存在且 raw_hash 变化且 user_modified=1，则不覆盖，并写一条 alert

(9) **AIAssistantManager**: 封装 DeepSeek 调用、prompt 模板、token 计数与对话历史。

4. Repository 层 (app/repositories/) —— 数据访问

这一层的定位: 它是“业务逻辑”和“数据库”之间的唯一中间人。上层的 Manager 想存一个任务、查一门课，无需自己写 SQL，只需调用 Repository 的方法；相应地，从数据库里存储的字符串和数字，到程序里的 Task、Course 等对象，两者之间的翻译工作也全部由 Repository 承担。这样设计的好处是：之后若更换数据库或表结构，只需改动 Repository，上层业务代码则无需更改。

Task / Course / Schedule / Exam / Alert / Sync / Setting 等每一类数据都配有一个专属仓库（如 TaskRepository、CourseRepository）。它们的接口统一，都提

供以下标准操作：

方法	作用	通俗解释
<code>add(obj)</code>	增	插入一条新记录，并把数据库自动生成的 id 回填给对象
<code>get_by_id(id)</code>	查单个	按 id 取一条，取不到返回 <code>None</code> （而不是报错）
<code>list_all()</code>	查全部	取出所有记录，通常已按截止时间排好序
<code>update(obj)</code>	改	按 id 覆盖整条记录，返回是否真的改到了
<code>delete(id)</code>	删	按 id 删除
<code>find_by_external_id(eid)</code>	按外部 ID 查	同步时用教学网的原始 ID 反查本地记录，判断“这条是不是已经存过了”

这样设计的好处是：开发时只要写好 TaskRepository 当模板，其余仓库模仿即可，提升了效率与准确率。

值得一提的设计细节（以 TaskRepository 为例）：

（1）add / update 在真正执行 SQL 之前，会逐字段校验数据是否合法——标题不能为空、优先级必须是 1/2/3、状态必须是 todo/doing/done/blocked、预计耗时不能是负数或无穷大……不合规的数据会被直接拦截。

（2）教学网同步来的任务不会被物理删除，而是打一个 is_hidden=1 的隐藏标记（soft_delete），日常查询自动过滤掉隐藏项。只有那些既隐藏又已经逾期的同步任务，才会在同步结束后被真正清理（purge_hidden_overdue）。这样避免了删掉后再一同步又显示出来的情况。此外 find_by_external_id 只匹配 source='sync' 的记录，防止用户手动建的任务被外部 ID 误命中。

5. 数据库层（app/database/）

整个应用的用户数据都存在本地一个 SQLite 文件里（data/ddl_center.db），不依赖任何服务器，在断网的情况下也能使用。

（1）**DatabaseManager**：数据库的“总管家”。负责建立连接、首次启动时读取 schema.sql 并自动建好所有表，提供数据库的备份与重置功能。

（2）**schema.sql**：一份“建表说明书”，把整个数据库结构集中写在一个文件里，一目了然。共 9 张表。其中含 7 张核心业务表：tasks（任务）、courses（课程）、schedule_slots（课表时段）、exams（考试）、alerts（提醒）、sync_records（同步记录）、user_settings（用户设置）和 2 张 AI 专用表：ai_conversations（对话会话）、ai_messages（每条聊天消息）。

（3）**两项性能与正确性保障**：

建索引提速：对最常被用来筛选/排序的字段（due_time、course_id、status、external_id）建立索引。任务数量变多后，“看某门课的作业”“按截止时间排序”等操作

作依然很快，不会随数据增长而变卡。

靠唯一约束防止数据重复：在 sync_records 表上设置唯一性约

束 UNIQUE(source_type, external_id)，从数据库层面确保"同一个外部对象"永远只有一条同步记录，即使代码逻辑偶有疏漏，也不会产生重复数据。

6. 网络与解析层 (app/network/、app/parsers/) —— 实现教学网同步

这两层配合完成把教学网数据搬到本地的全过程。

(1) AuthClient (登录)：北大教学网使用 IAAA 统一身份认证，登录需要走**四步**：先向 IAAA 要一把 RSA 公钥，之后用这把公钥把密码加密，再带着加密后的密码 POST 到 oauthlogin.do，换回一个一次性 token，最后拿 token 访问 campusLogin 完成跳转，会话里就自动带上了教学网的 Cookie。

登录成功后返回一个 AuthSession 对象，里面装着这把"带通行证的会话"，后续所有请求直接复用它即可。

(2) TeachingSiteClient (抓取)：用登录好的会话去请求作业页、课表页、考试页，把 HTML / JSON 原文原封不动地拿回来，但不解析。这样抓取和解析解耦，日后教学网页面改版，只需改解析层。

(3) Parsers (解析)：DDLParser (作业)、ScheduleParser / PortalScheduleParser (课表)、ExamParser (考试) 各自用 BeautifulSoup 读取对应网页的 DOM 结构，把杂乱的 HTML 抽取成干净的 Task / ScheduleSlot / Exam 模型对象，并交给上层入库。

(4) 分层的异常体系：网络环节可能出各种错，我们用一套继承树将它们分类——基类 NetworkError 之下有 AuthError (账号密码错、认证失败)、TokenExpiredError (token 过期，属于 AuthError 的一种)、ConnectionError (断网、超时)、ParseError (返回内容不是合法的 HTML/JSON)、SyncError (登录之后同步过程中的其他错误)。这样上层可以依据错误类型精准处理：能重试的重试，实在连不上的就 fallback 到本地 mock 数据继续演示，而不会全盘崩掉。

7. GUI 层 (app/gui/) —— 交互部分

界面使用 PySide6 建成，采用"一个主窗口 + 多个专职组件"的组织方式。

(1) 主窗口：作为整个界面的总调度台，把任务列表、提醒面板、课程表、统计、AI 每日简报等各个功能区组织在一起，并提供彼此跳转的入口。

(2) 各司其职的子组件：TaskEditorDialog (新增/编辑任务的弹窗)、TaskListWidget (任务列表，含时间轴/课程分组/状态筛选)、ScheduleWidget (课程表)、StatisticsWindow (进度统计图表)、SyncDialog (登录与同步)、AIChatDialog (AI 聊天窗口)、SettingsDialog (设置 API Key、主题等)。每个组件互不干扰。

(3) 后台线程：网络同步、AI 请求这类操作往往要等好几秒，如果直接放在按钮的点击响应里执行，整个界面会卡住（点击时无响应）。为此，所有耗时操作统一交给 qt_workers 提供的 SyncWorker / AIWorker 拿到后台线程里去跑，跑完再通过信号把结果送回界面刷新。这套线程模板由后端统一提供，界面开发者直接拿来用，不必自己处理复杂的多线程细节。

8. 几个有代表性的实现难点

难点一：DDL 红线要能在课表上悬浮

课程表是使用 Qt 的网格布局画成的，一个格子只能整块地放一个控件，格子边界是固定的。但"DDL 红线"和"当前时间蓝线"要体现真实的时刻——比如 10:37 的红线，它并不刚好卡在某个格子的边上，而是落在某一格的中间。

我们的解决办法是：在整张课表网格上面盖一层透明的画布，红线蓝线直接根据相应时刻对应第几个像素用绝对坐标画在这层透明画布上。思路虽然不难，但在实现的过程中遇到了以下问题：

1. 界面还没排版完时去问控件坐标，拿到的是 0×0（相当于线画到了左上角）；
2. 窗口第一次显示时不会触发"尺寸变化"条件，线不会自动重画；
3. 定时器每分钟刷新时，布局可能还没重新算过。

我们最终在代码里加了针对性的重算与补偿措施，解决了这些问题。

难点二：“凌晨三点”的线该画在哪

课表的可见范围只有 8:00 - 21:30。可是用户的 DDL 可能是 23:59，看课表的时间也可能是凌晨 03:00——这些时刻都在课表范围之外。

我们的处理办法是：若时间早于第一节课，把线画在课表最顶端；若时间晚于最后一节课，把线画在课表最底端；若时间落在两节课的空隙里，则按比例做线性插值，让线落在两节课之间的合理位置。

难点三：AI（DeepSeek）成本控制

我们在调用前后设了三道关卡：

1. 预算闸：发请求之前，用"字符数 ÷ 4"粗略估算这次要花多少 token，超出当日预算就直接拦下；
2. 缓存闸：同一条 DDL 描述按内容算一个 SHA256 指纹，一小时内重复遇到就直接用上次的摘要结果——同一个作业在多次同步里只花一次钱；
3. 短路闸：如果描述本身很短（≤120 字），就直接截断当摘要用，没必要为一句话去调 AI。

三闸叠加后，日常同步加聊天的花费稳定在每天几千 token 以内。

难点四：让 AI 在不获取整个数据库的情况下了解我的日程

直接把“我下周三有空吗”这句话发给 AI，AI 是答不上来的——它需要先知道你的任务和课表。但如果把所有任务、整学期课表全发过去，token 会瞬间爆表。

我们的做法是每轮对话发送一份精心裁剪过的“世界快照”：未完成任务按紧急程度排序后只取 8-12 条，课表只取当前这一周，如果某天时段超过 80 个就自动折叠成“其余 N 个已省略”。这样 AI 既能看到回答问题所必需的信息，单轮消耗又有明确上限。

三、小组成员分工情况

在项目开发层面，通过 AppFacade / Repository 接口解耦并行开发。杨恩华负责后端、网络解析和 AI 部分，郭浩充负责 GUI 全部窗口与组件，陈伊扬负责数据库和 Repository 部分。

在总结汇报层面，杨恩华和郭浩充负责程序运行、应用讲解的视频录制，陈伊扬负责作业报告的撰写。

四、项目总结与反思

1. 项目成果与创新点

DDL-Center 的核心创新不在于单独实现一个任务列表或课程表，而在于把北大学生真实学习场景中分散的信息源整合到同一个本地工作台。教学网作业、门户课表、考试安排、个人 TODO、提醒预警和 AI 助手原本分散在不同平台与工具里，用户需要反复切换、手动记录和自行判断轻重缓急。本项目将这些信息统一收束到一个桌面应用中，使“查看任务—理解日程—判断风险—安排时间”形成连续流程。

项目的第一个亮点是面向北大场景的深度适配。系统支持 IAAA 登录与教学网作业同步，也支持从门户 HTML 中导入课表和考试信息。这不是通用待办软件能够直接覆盖的需求，而是针对校内平台、校内数据格式和学生实际使用习惯设计的功能。通过自动同步和解析，项目减少了手动抄写 DDL 与维护课表的负担。

第二个亮点是将 DDL 管理和课表视图结合起来。传统 TODO 工具通常只列出截止时间，而课程表工具通常只显示上课安排。本项目把任务、课程、考试和当前时间线叠加到同一视角中，让用户不仅知道“有哪些任务”，还知道“这些任务和本周时间安排如何冲突”。红色 DDL 线、当前时间线、同日超载预警和相邻 DDL 拥堵预警，使压力分布变得更加直观。

第三个亮点是 AI 的务实使用。项目没有把 AI 当作核心业务逻辑的替代品，而是将其放在自然语言录入、任务拆解、同步内容摘要、每日简报和对话建议等更适合大模型发挥作用的环节。同时，系统设计了 token 预算、摘要缓存、上下文裁剪和本

地 fallback，保证 AI 不可用时应用仍然可以完成核心功能。这种设计避免了“为了 AI 而 AI”，而是让 AI 成为已有学习管理流程的增强层。

第四个亮点是同步过程对用户数据的保护。教学网同步并不是简单地覆盖本地数据，而是以 external_id 去重，并保留用户手动编辑过的任务信息。这样既能享受自动同步的便利，又不会因为重新同步而丢失用户已经补充或修改过的内容。对于一个面向长期使用的本地工具来说，这种“不破坏用户数据”的设计非常重要。

第五个亮点是本地优先。项目数据存储在本地的 SQLite 中，核心任务管理、课表查看、提醒和统计功能不依赖服务器。对于学生个人工具而言，本地存储降低了部署和隐私成本，也使项目更容易演示、迁移和维护。

2. 困难与反思

首先，Qt 布局和绘制生命周期比预期复杂。课表红线和当前时间蓝线需要在网格布局上做像素级定位，开发过程中遇到了首次显示尺寸未计算、resize 未触发、定时刷新时布局未稳定等问题。这说明 GUI 开发不能只关注控件摆放，还必须理解框架的绘制时机和事件模型。

其次，外部系统同步具有天然不确定性。教学网和门户页面不是稳定公开 API，HTML 结构、认证流程或字段格式变化都可能影响解析结果。因此在开发过程中，我们不仅要实现“能抓取”，还要考虑异常处理、降级策略和解析回归测试，否则功能很容易因为外部页面变化而失效。

第三，AI 功能必须被严格约束。大模型能够提升自然语言录入和摘要体验，但也可能误判任务类型或编造截止时间。项目中通过无截止时间不入库、本地 fallback、上下文裁剪等方式降低风险，但这一过程也提醒我们：AI 更适合作为辅助判断工具，而不能无条件信任其输出。

第四，协作流程后期执行不够稳定。开发后期由于任务较重，没有始终坚持每个新功能单独开 branch、及时 review、及时合并的工作流，导致 Git 历史较混乱，也增加了定位问题的成本。这个问题说明，越到项目后期，越需要保持小步提交和清晰分支，而不是为了赶进度牺牲协作规范。

3. 总结

总体而言，DDL-Center 已经形成了一个功能完整、可稳定演示、具有真实使用价值的成品。它的价值不只是实现了任务管理、课表导入或 AI 对话等单项功能，而是围绕北大学子的 DDL 管理痛点，构建了一个从信息获取、任务整理、风险提示到时间安排的完整闭环。通过这个项目，我们不仅提升了桌面应用、数据持久化、网络解析和 AI 接入等技术能力，也更深入地理解了多人协作、接口设计、用户数据保护和工程化测试的重要性。